Next.js automatically optimizes fonts in the application when you use the next/font module. It downloads font files at build time and hosts them with your other static assets. This means when a user visits your application, there are no additional network requests for fonts, which would impact performance.

==========

To add a new font, first create font.ts:

```
import { Inter } from 'next/font/google'

export const inter = Inter({ subsets: ['latin'] })
```

Then import it at the root layout:

```
import '@/app/ui/global.css';

import { inter } from '@/app/ui/fonts';

export default function RootLayout({
 children,
}: {
 children: React.ReactNode;
}) {
 return (
  <html lang="en">
   <body className={`${inter.className} antialiased`}>{children}</body>
  </html>
 );
}
```

==========

With regular HTML, you would add an image as follows:

```
<img
  src="/hero.png"
  alt="Screenshots of the dashboard project showing desktop version"
/>
```

However, this means you have to manually:

- Ensure your image is responsive on different screen sizes.
- Specify image sizes for different devices.
- Prevent layout shift as the images load.
- Lazy load images that are outside the user's viewport.

==========

Image Optimization is a large topic in web development that could be considered a specialization in itself. Instead of manually implementing these optimizations, you can use the **next/image** component to automatically optimize your images.

==========

The `<Image>` component

The `<Image>` Component is an extension of the HTML `<img>` tag, and comes with automatic image optimization, such as:

- Preventing layout shift automatically when images are loading.
- Resizing images to avoid shipping large images to devices with a smaller viewport.
- Lazy loading images by default (images load as they enter the viewport).
- Serving images in modern formats, like WebP and AVIF, when the browser supports it.

==========

Images without dimensions and web fonts are common causes of layout shift due to the browser having to download additional resources.

==========

```
import Image from 'next/image';

<div className="flex items-center justify-center p-6 md:w-3/5 md:px-28 md:py-12">
      {/* Add Hero Images Here */}
      <Image
       src={"/hero-desktop.png"}
       width={1000}
       height={760}
       className='hidden md:block'
       alt="Screenshots of the dashboard project showing desktop version"
      />
      <Image
       src="/hero-mobile.png"
       width={560}
       height={620}
       className='block md:hidden'
       alt="Screenshots of the dashboard project showing mobile version"
      />
</div>
```

==========

Here, you're setting the **width** to 1000 and **height** to 760 **pixels**. It's good practice to set the width and height of your images to avoid layout shift; these should be an aspect ratio **identical** to the source image. These values are *not* the size the image is rendered, but instead the size of the actual image file used to understand the aspect ratio.

==========

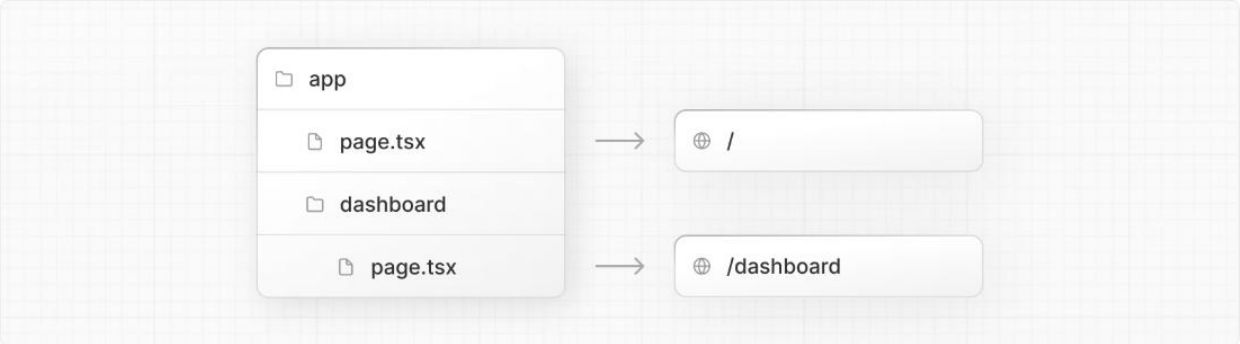
Next.js uses file-system routing, where **folders** are used to create nested routes. Each folder represents a **route segment** that maps to a **URL segment**.

==========

You can create separate UIs for each route using **layout.tsx** and **page.tsx** files. The **page.tsx** is a special Next.js file that exports a React component, and it's required for the route to be accessible.

==========

To create a nested route, you can nest folders inside each other and add **page.tsx** files inside them.



`/app/dashboard/page.tsx` is associated with the `/dashboard` path. Let's create the page to see how it works!

==========

The layout file is the best way to create a shared layout that all pages in your application can use.

==========

The flex: none is equivalent to flex: 0 0 auto, which is shorthand for:

- **flex-grow**: 0
- **flex-shrink**: 0
- **flex-basis**: auto

Simply put, **flex: none** sizes the flex item according to the width/height of the content, but doesn't allow it to shrink. This means the item has the potential to overflow the container.

If you omit the flex property (and longhand properties), the initial values are as follows:

- **flex-grow**: 0
- **flex-shrink**: 1
- **flex-basis**: auto

This is known as **flex: initial**.

This means the item will not grow when there is free space available (**just like flex: none**), but it can shrink when necessary (**unlike flex: none**).

==========

The layout file is the best way to create a shared layout that all pages in your application can use.

==========

Why optimize navigation?

To link between pages, you'd traditionally use the `<a>` HTML element. At the moment, the sidebar links use `<a>` elements, but notice what happens when you navigate between the home, invoices, and customers pages on your browser.

Did you see it?

There's a full page refresh on each page navigation!

==========

The Link component is similar to using <**a**> **tags**, but instead of <**a** href="...">, you use <**Link** href="...">.

==========

Automatic code-splitting and prefetching

To improve the navigation experience, Next.js automatically code splits your application by route segments. This is different from a traditional React SPA ↗, where the browser loads all your application code on the initial page load.

Splitting code by routes means that pages become isolated. If a certain page throws an error, the rest of the application will still work. This is also less code for the browser to parse, which makes your application faster.

Furthermore, in production, whenever `<Link>` components appear in the browser's viewport, Next.js automatically **prefetches** the code for the linked route in the background. By the time the user clicks the link, the code for the destination page will already be loaded in the background, and this is what makes the page transition near-instant!

==========

Next.js automatically prefetches the code for the linked route in the background. By the time the user clicks the link, **the code for the destination page will already be loaded in the background**, and this is what makes the page transition near-instant!

==========

When using the path name hook, we should convert the component to client:

```
'use client'
import { usePathname } from 'next/navigation';

export default function NavLinks() {
  const pathname = usePathname()
  return (
    <>
      {links.map((link) => {
        const LinkIcon = link.icon;
        return (
          <Link
            key={link.name}
            href={link.href}
            className={
              clsx(
                "flex h-[48px] grow items-center justify-center gap-2 rounded-md
bg-gray-50 p-3 text-sm font-medium hover:bg-sky-100 hover:text-blue-600 md:flex-
none md:justify-start md:p-2 md:px-3",
                {
                  'bg-sky-100 text-blue-600': pathname === link.href,
                }
              )
            }
          >
            <LinkIcon className="w-6" />
            <p className="hidden md:block">{link.name}</p>
          </Link>
        );
      })}
    </>
  );
}
```

==========

Using Server Components to fetch data

By default, Next.js applications use **React Server Components**. Fetching data with Server Components is a relatively new approach and there are a few benefits of using them:

- Server Components support JavaScript Promises, providing a solution for asynchronous tasks like data fetching natively. You can use `async/await` syntax without needing `useEffect`, `useState` or other data fetching libraries.
- Server Components run on the server, so you can keep expensive data fetches and logic on the server, only sending the result to the client.
- Since Server Components run on the server, you can query the database directly without an additional API layer. This saves you from writing and maintaining additional code.

==========

In this way, we can use async with the server component pages:

```
export default async function Page() {
    return (
        <main>
            <h1 className={`${lusitana.className} mb-4 text-xl md:text-2xl`}>
                Dashboard
            </h1>
            <div className="grid gap-6 sm:grid-cols-2 lg:grid-cols-4">


            </div>
            <div className="mt-6 grid grid-cols-1 gap-6 md:grid-cols-4 lg:grid-
cols-8">

            </div>
        </main>
    )
}
```

==========

By default, Next.js **prerenders** routes to improve performance, which is called **Static Rendering**. So, if your data changes, it won't be reflected in your dashboard.

==========

A "**waterfall**" refers to a sequence of network requests that **depend on the completion of previous requests**. In the case of data fetching, each request can only begin once the previous request has returned data.

==========

What is Static Rendering?

With static rendering, data fetching and rendering happens on the server at build time (when you deploy) or when revalidating data.

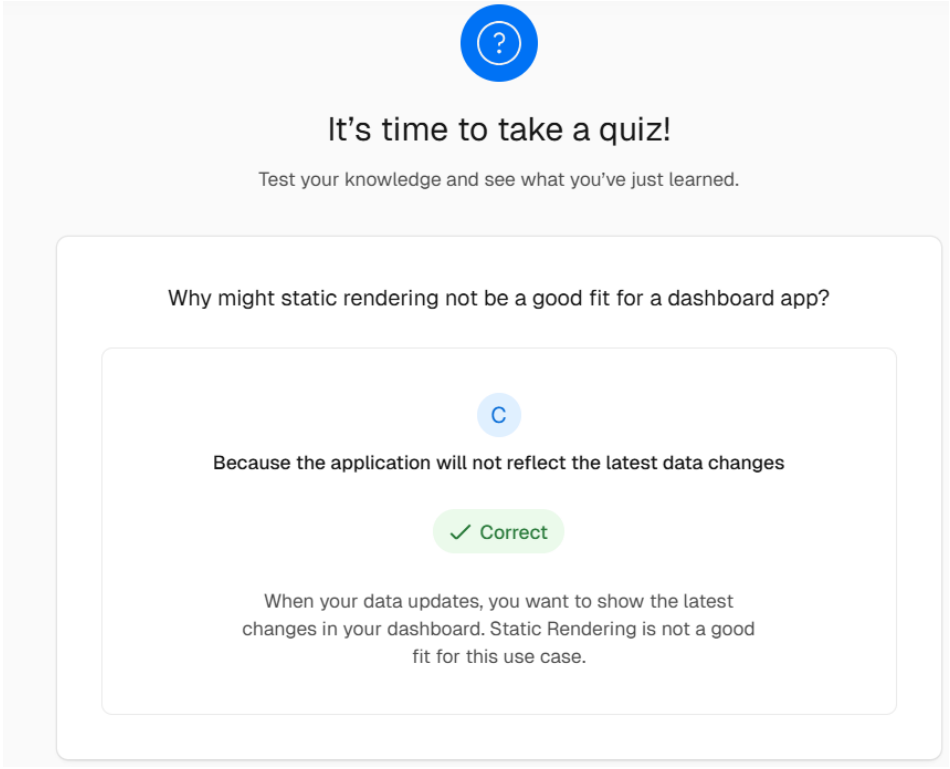
Whenever a user visits your application, the cached result is served. There are a couple of benefits of static rendering:

- **Faster Websites** - Prerendered content can be cached and globally distributed when deployed to platforms like Vercel ↗. This ensures that users around the world can access your website's content more quickly and reliably.
- **Reduced Server Load** - Because the content is cached, your server does not have to dynamically generate content for each user request. This can reduce compute costs.
- **SEO** - Prerendered content is easier for search engine crawlers to index, as the content is already available when the page loads. This can lead to improved search engine rankings.

Static rendering is useful for UI with **no data** or **data that is shared across users**, such as a static blog post or a product page. It might not be a good fit for a dashboard that has personalized data which is regularly updated.

The opposite of static rendering is dynamic rendering.

==========


It’s time to take a quiz!
Test your knowledge and see what you’ve just learned.
Why might static rendering not be a good fit for a dashboard app?
C
Because the application will not reflect the latest data changes
Correct
When your data updates, you want to show the latest changes in your dashboard. Static Rendering is not a good fit for this use case.

==========